סדנה מתקדמת בתכנות

פרק 4 : רשימות מקושרות

למה אנו צריכים את זה?

- נניח שרוצים לכתוב תוכנה לניהול הספרים בספריה, או לניהול מלאי של סופרמרקט.

- מה שמאפיין מערכות כאלה הוא תחלופה תכופה יחסית: ספרים חדשים נקנים וישנים הולכים לאיבוד, מגוון המוצרים של הסופר משתנה באופן תכוף.

- התוכנות האלה צריכות להחזיק רשימת ספרים/מוצרים. הרשימה גודלת כשיש תוספת, וכשיש הפחתה צריך להקטין אותה.

למה אנו צריכים את זה?

- ניתן להחזיק רשימה כזאת במערך דינאמי, שנקצה מחדש בגודל גדול יותר בעת הצורך.

- אבל אז בכל תוספת עלולים להעתיק את כל המערך למקום חדש וגדול יותר בזיכרון (למשל, באמצעות פונקצית realloc) **וזה מאוד לא יעיל.**

- כמו-כן, אם למשל ספר הולך לאיבוד נצטרך למעשה "לצופף" את כל המערך כדי לבטל את הספר הזה. זה שוב מאוד לא יעיל.

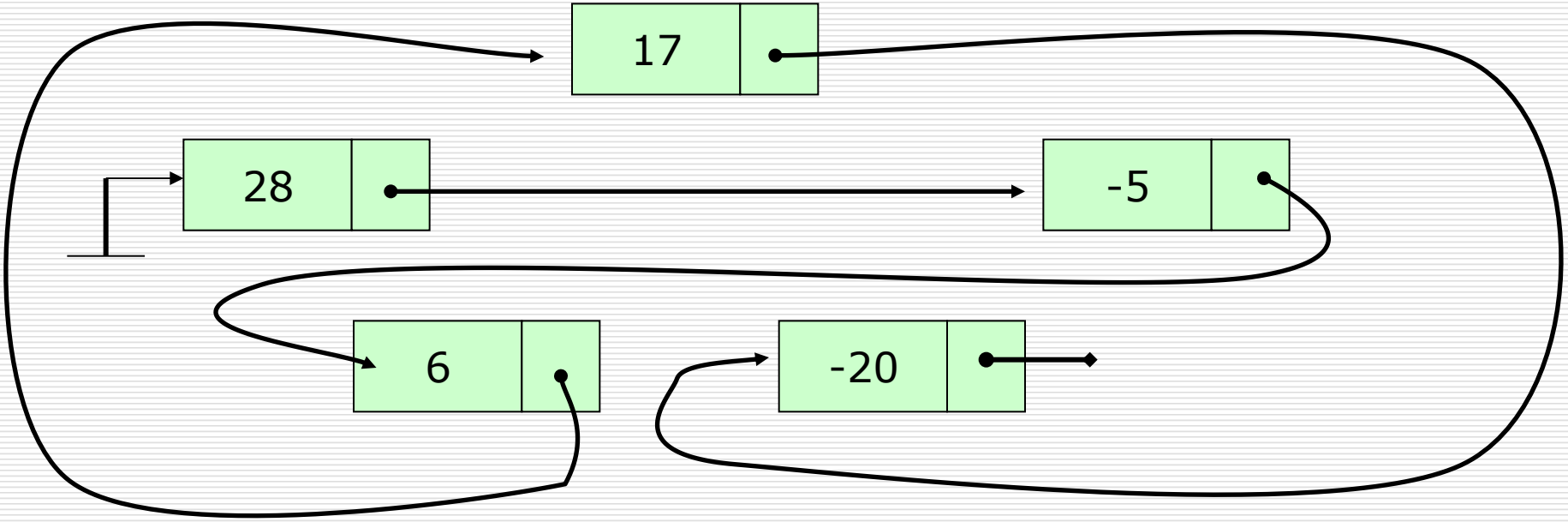
למה אנו צריכים את זה?

- אז גם מערך דינאמי לא פותר את הבעיה.
- כן, אפשר בכל שלב התוכנית להקצות ולשחרר מערך דינאמי, אבל כאשר הוא כבר קיים, הגודל שלו הוא קבוע ולא ניתן לשנות מספר האיברים במערך.
- ואפילו אם להניח שמוכן במערך מראש מספיק מקום לאיברים נוספים (אבל זה כבר בזבוז זיכרון!) איך להוסיף או למחוק איבר לא בסוף אלא באמצע או בתחילת המערך בלי הזזת איברים אחרים שלו?
- בעיה נוספת: מה אם רשימת הנתונים כל כך ארוכה שאין מספיק **תאים רצופים** בשבילה בזיכרון (ככה מאורגן מערך) למרות שבסה"כ יש מקום לכל הנתונים (**לא ברציפות**)?

למה אנו צריכים את זה? הצעה לפתרון

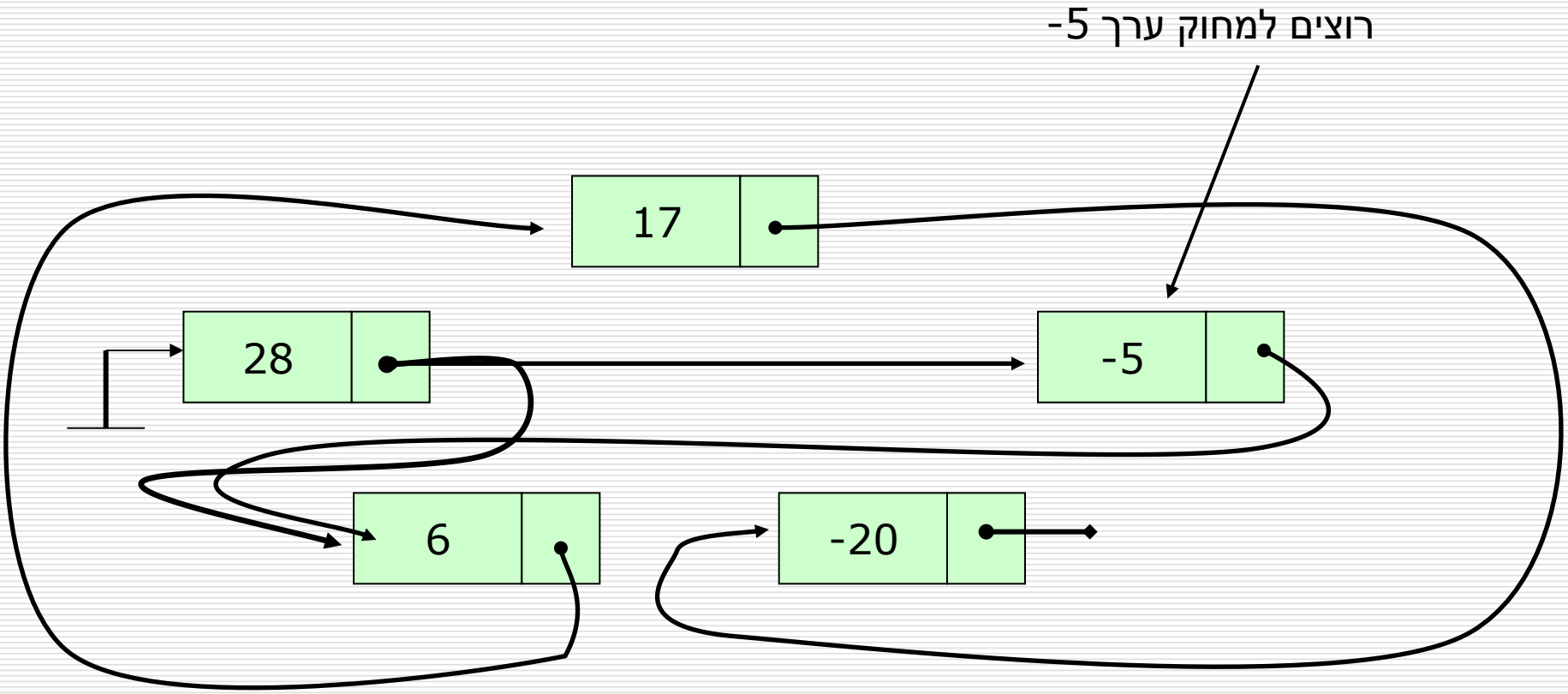
- לעבודה עם סדרות גדולות שיש בהן שינויים תכופים משתמשים ב- **רשימה מקושרת** במקום להשתמש במערך.

- רעיון לשיטה: (הוספת איבר חדש)



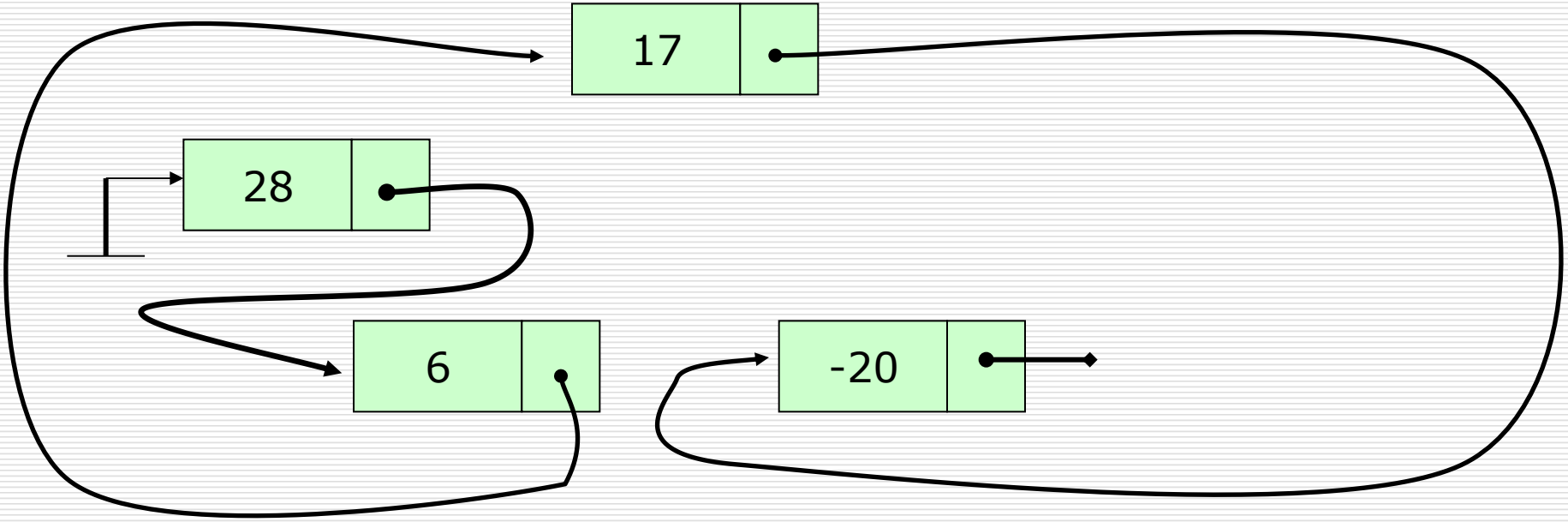
למה אנו צריכים את זה? הצעה לפתרון

- רעיון לשיטה: (מחיקת איבר)



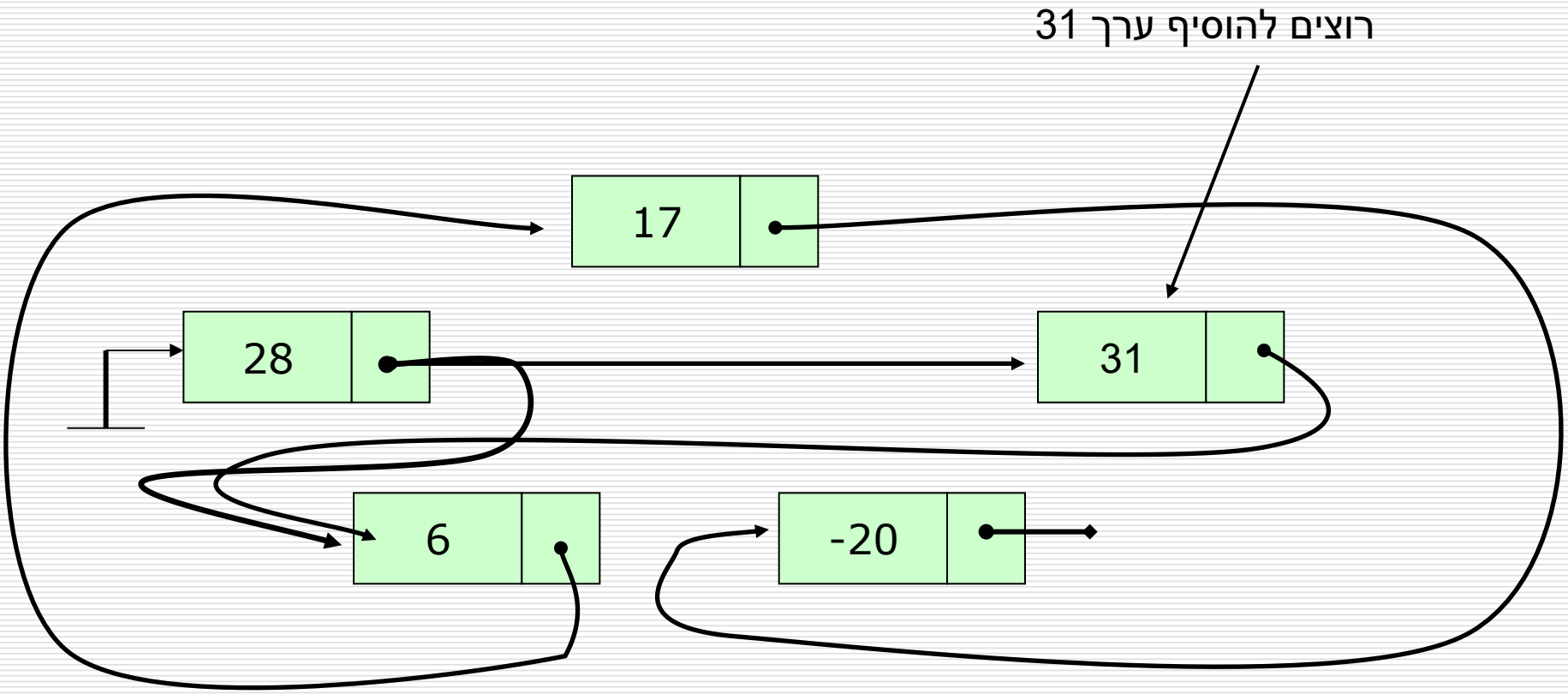
למה אנו צריכים את זה? הצעה לפתרון

- רעיון לשיטה: (מחיקת איבר)



למה אנו צריכים את זה? הצעה לפתרון

- רעיון לשיטה: (הוספת איבר חדש באמצע)



סיכום

- בניגוד למערך, *שהוא אוסף של תאים רצופים*, רשימה מקושרת *היא אוסף של תאים לא רצופים*, אשר כל אחד מהם **כולל גם מצביע לתא שאחריו ברשימה**.

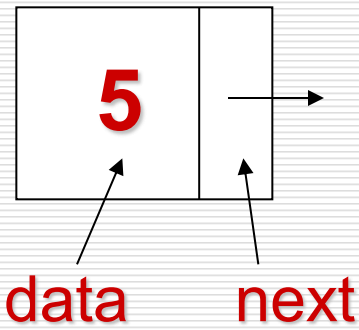
- כדי לעבור על הרשימה מספיק להחזיק מצביע לתא הראשון שלה, וממנו להתקדם לתא הבא, וכן הלאה.

- המוסכמה היא שהתא האחרון ברשימה כולל מצביע NULL.

מימוש איברי רשימות מקושרות

- איבר ברשימה ייוצג על-ידי מבנה שיכיל לפחות שני שדות: לפחות שדה אחד יכיל מידע (בדוגמה שלנו יש שדה אחד שמכיל מספר שלם), ושדה אחד יכיל מצביע לאיבר הבא.

```
typedef struct item{
    int data;
    struct item *next;
}Item;
```



- השדה data יכיל את התוכן של האיבר.
- השדה next יכיל מצביע לאיבר הבא ברשימה.

(אם אין איבר נוסף, כלומר זה האיבר האחרון, אז ערך המצביע הזה יהיה NULL)

רשימות מקושרות ב-C

- שימו לב שהמצביע next מצביע על האיבר הבא, שהוא מאותו סוג כמו האיבר הנוכחי (כי כל איברי הרשימה הם מאותו סוג).

- מותר שבהגדרתו המבנה יכלול מצביע למבנה מאותו סוג: הקומפיילר מסתדר עם זה כי מבחינת ההגדרה מספיק לו לדעת שתאוחסן שם כתובת.
- חייבים לתת שם ל-struct כבר מעל תוכן שלו!
- נתאר איך לממש פעולות על רשימות בעזרת המבנה הזה (הוספה, מחיקה, וחיפוש של איברים).

רשימות מקושרות ב-C

- בקורס "מבני נתונים" רשימה מקושרת היא טיפוס מופשט.
- בשפת C רשימה מקושרת היא פשוט שיטה לאחסון נתונים.
- לכן בהתחלה לא נממש כל הפעולות על רשימות מקושרות באמצעות פונקציות נפרדות.
- נכתוב פונקציה לבניית רשימה מקושרת.
- על הפונקציה לקלוט תווים ממקלדת ולהוסיף כל תו חדש לסוף הרשימה עד ללחיצה על Enter.
- בתחילת בניית הרשימה לא ידוע מה יהיה גודל שלה ולכן מערך דינאמי לא מתאים פה.

בניית רשימה מקושרת

```
typedef struct item
{
      char data;
      struct item *next;
}Item;
```

```
Item *build_char_list ()
{
    Item *lst, *curr_point;
    char c;

     if ((c=getchar())=='\n')
          return NULL;
     // processing of the first item
     lst = (Item *)malloc(sizeof(Item));
     curr_point = lst;
     lst->data = c;
     // processing of the second and other items
    while ((c=getchar())!='\n')
     {
          curr_point->next = (Item *)malloc(sizeof(Item));
          curr_point = curr_point->next;
          curr_point->data = c;
     }
    curr_point->next=NULL;
    return lst;
}
```

ספירת איברים עם תוכן נתון ברשימה

```
int numb_in_list (Item *lst, char key)
{
    int n=0;

    while (lst)
    {
        if (lst->data == key)
            n++;
        lst = lst->next;
    }
    return n;
}
```

שימוש בפונקציות

```
void main()
{
    Item *char_list;
    int n;

    char_list =  build_char_list ();
    n = numb_in_list (char_list, 'x');
    .
    .
    .
}
```

אתחל רשימה

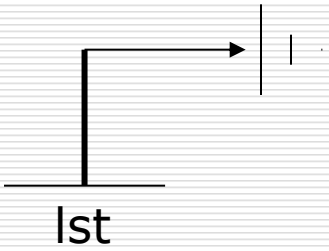
- כדי לשמור רשימה מקושרת, מספיק לשמור רק כתובת האיבר הראשון.

```
Item *lst;   // מחזיק תחילת הרשימה
```

- כדי לגלוש תוך רשימה יהיה מספיק לקבוע מצביע ל-Item אשר מתחיל מכתובת lst.

- אתחול רשימה – רשימה ריקה:

```
lst=NULL;
```



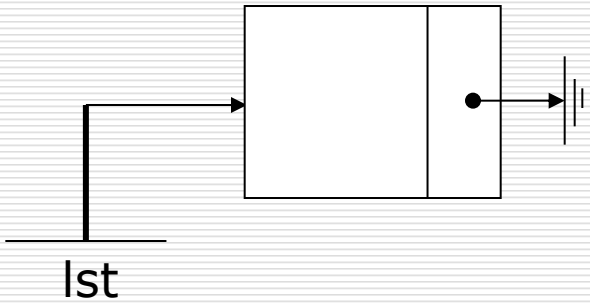
איבר דמה – dummy element

- האיבר הראשון מטופל באופן מיוחד במהלך בנית הרשימה.
- לשם נוחות ניתן (אבל לא חובה!) להוסיף **איבר דמה** שיצביע על האיבר הראשון, ולא להתחיל את הרשימה ישר מאיבריה

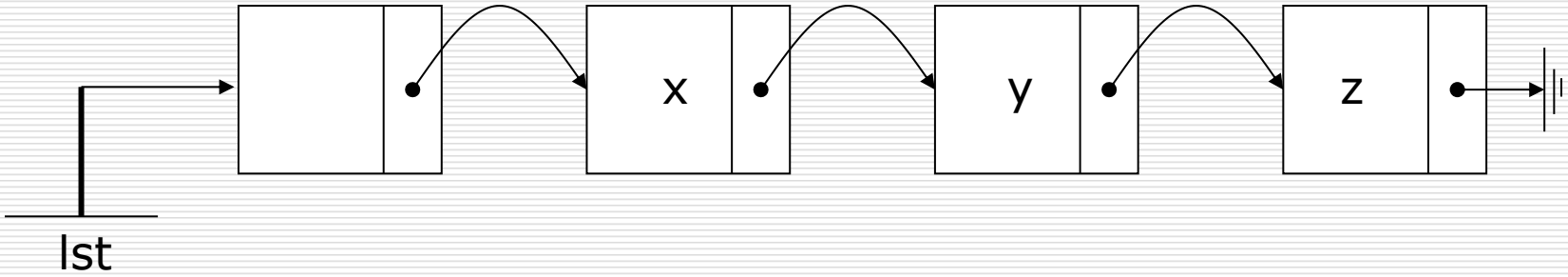
```
Item *lst;  // מחזיק תחילת הרשימה
```

- אתחול רשימה – רשימה ריקה:

```
lst = (Item *)malloc(sizeof(Item));
lst->next = NULL;
```

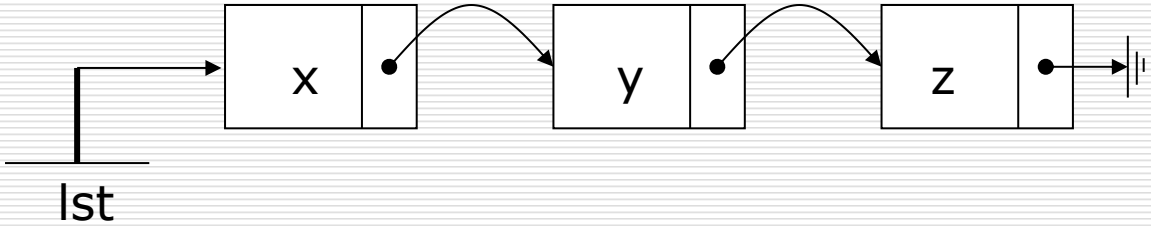


רשימה עם איבר דמה



- אחרי הבניה (אם ידוע שברשימה כבר לא יהיו שינוים) ניתן (אבל לא חובה!) למחוק **איבר דמה** :

```
curr_point = lst;
lst = lst->next;
free (curr_point);
```



- מי שיעבוד לאחר מכן עם הרשימה צריך לדעת האם יש או אין **איבר דמה** בתחילת הרשימה!

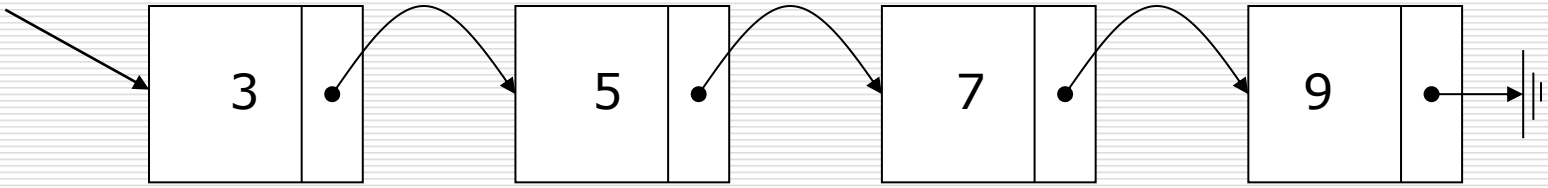
מימוש פעולות בסיסיות על רשימה מקושרת

- מימוש פעולות על רשימה מקושרת ב- C :
  - הוספת איבר.
  - מחיקת איבר.
  - מציאת איבר.
  - מחיקת כל הרשימה.

```
typedef int list_type;

typedef Item* list;
```

דוגמת ניגוד למערך

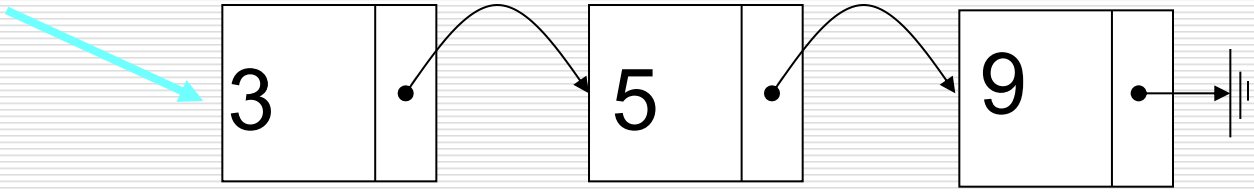


לעומת מערך:

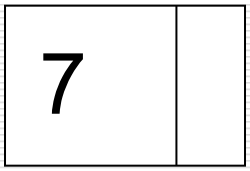
3	5	7	9

דוגמא בציור: הוספת איבר בהתחלה

- נניח שנתונה הרשימה הבאה של מספרים ורוצים להוסיף בהתחלה איבר עם התוכן 7:

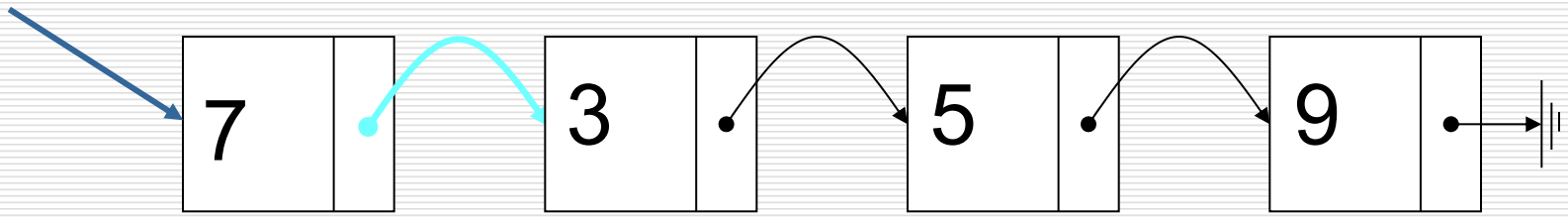


- אז ניצור איבר אחד חדש עם התוכן 7:



רשימות מקושרות - הוספת איבר בהתחלה

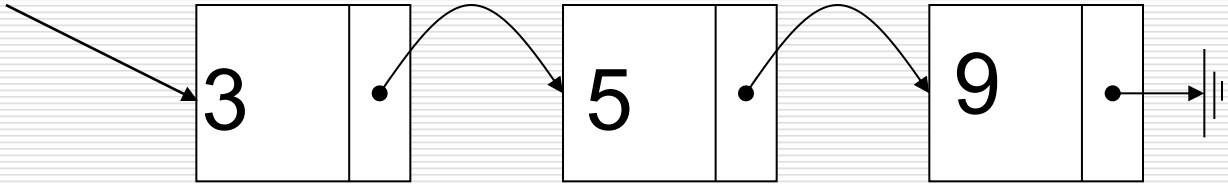
- נדאג לכך שהאיבר 7 יצביע לראש הרשימה המקורית, וראש הרשימה החדש שלנו יהיה האיבר 7.



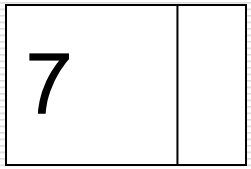
- במימוש על-ידי מערך היינו צריכים להזיז קדימה את כל הערכים הקיימים (בהנחה שהיה מקום)

עוד דוגמא בציור: הוספת איבר בסוף

- נניח שנתונה הרשימה הבאה של מספרים ורוצים להוסיף בסופה איבר עם המספר 7:

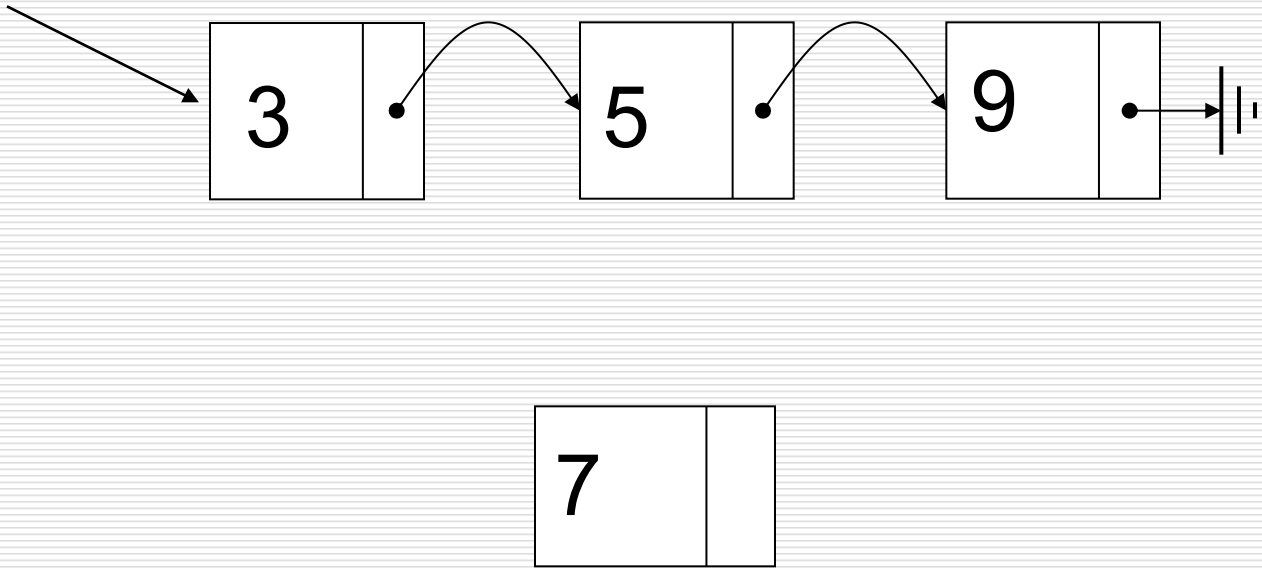


- אז ניצור איבר נוסף:



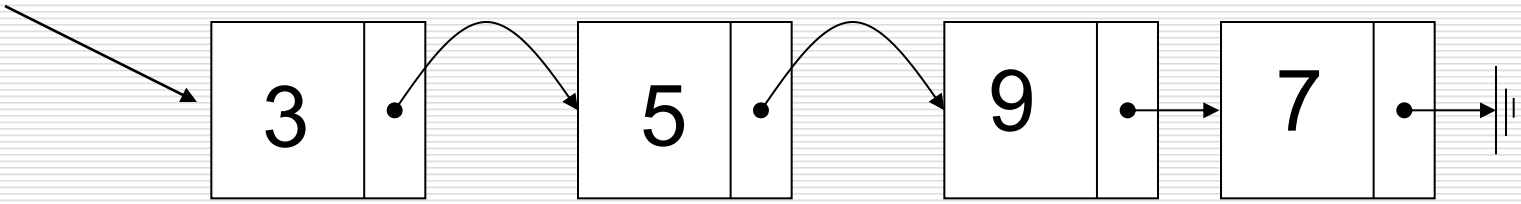
רשימות מקושרות - הוספת איבר בסוף

- נחפש את האיבר האחרון ברשימה, וכשנמצא אותו נגרום לו להצביע על 7. 7 יהפוך להיות האיבר האחרון.



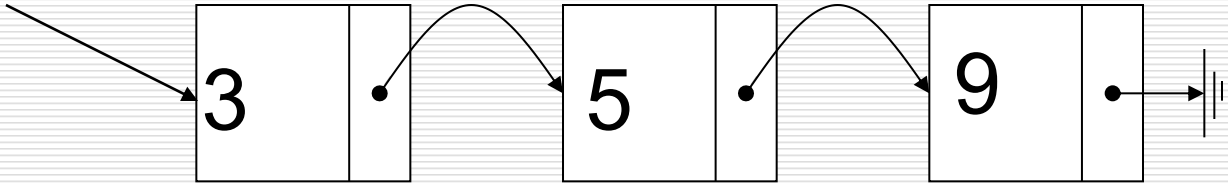
רשימות מקושרות - הוספת איבר בסוף

- נחפש את האיבר האחרון ברשימה, וכאשר נמצא אותו נגרום לו להצביע על 7, ו-7 יהפוך להיות האיבר האחרון.

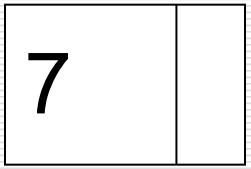


עוד דוגמא בציור: הוספת איבר באמצע

- נניח שנתונה הרשימה הבאה של מספרים ורוצים להוסיף איבר עם המספר 7 בין איברי 5 ל-9:

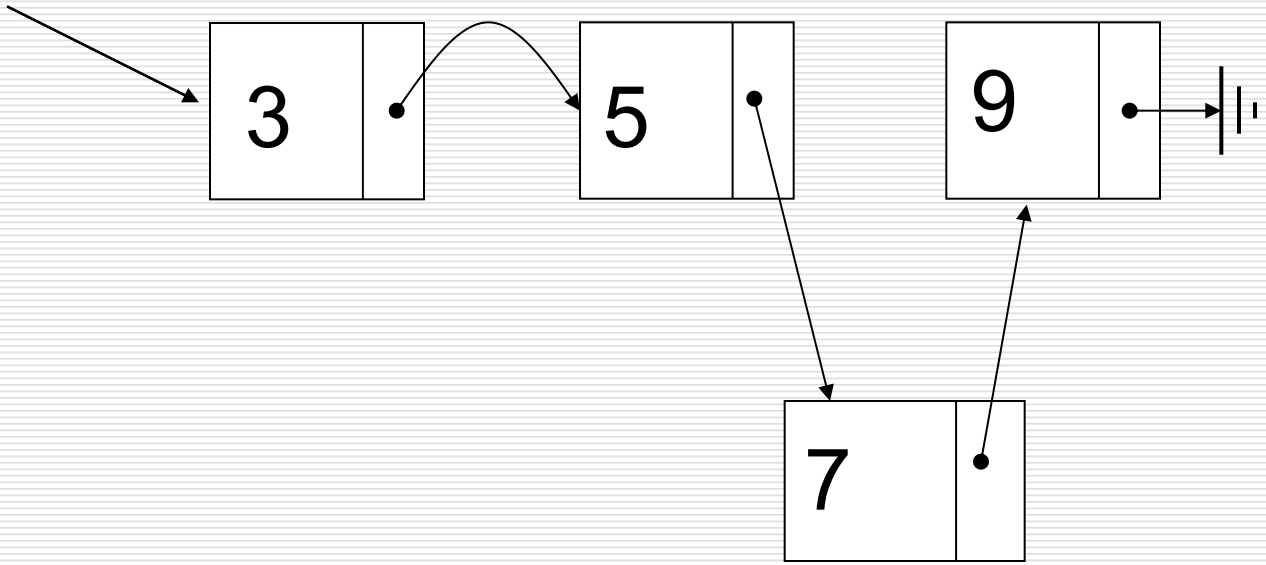


- אז ניצור איבר נוסף:



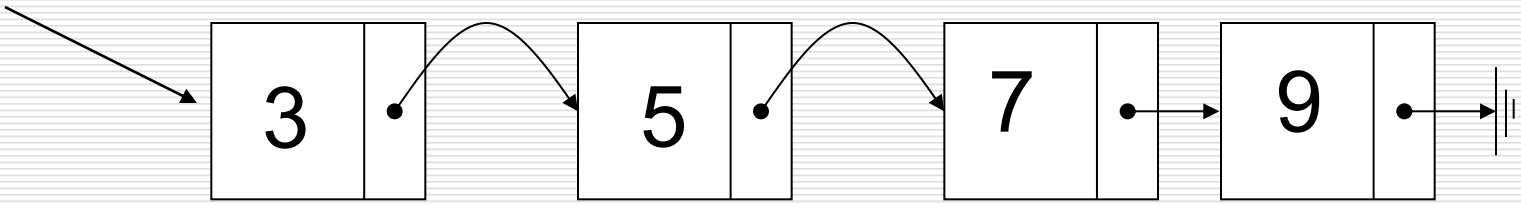
רשימות מקושרות - הוספת איבר באמצע

- נחפש את האיבר 5 ברשימה, וכשנמצא אותו נגרום לו להצביע על 7. נדאג לכך שהאיבר 7 יצביע על 9.



רשימות מקושרות - הוספת איבר באמצע

- נחפש את האיבר 5 ברשימה, וכשנמצא אותו נגרום לו להצביע על 7. נדאג לכך שהאיבר 7 יצביע על 9.



- במימוש על-ידי מערך היינו צריכים להזיז קדימה את כל הערכים אחרי 5 (בהנחה שהיה מקום)

הוספת איבר חדש

```
Item *Insert (list_type x, Item *pos, list *pl)
 {
    Item *temp = (Item *)malloc(sizeof(Item));
    if (!temp) return NULL;
    temp->data = x;
    if (!pos || !(*pl)) // insert into head or list is empty
    {
        temp->next = *pl;
        *pl = temp;
     }
    else // insert into middle (or end)
    {
        temp->next = pos->next;
        pos->next = temp;
     }
    return temp;
 }
```

מוסיפים איבר חדש אחרי
איבר עליו מצביע pos

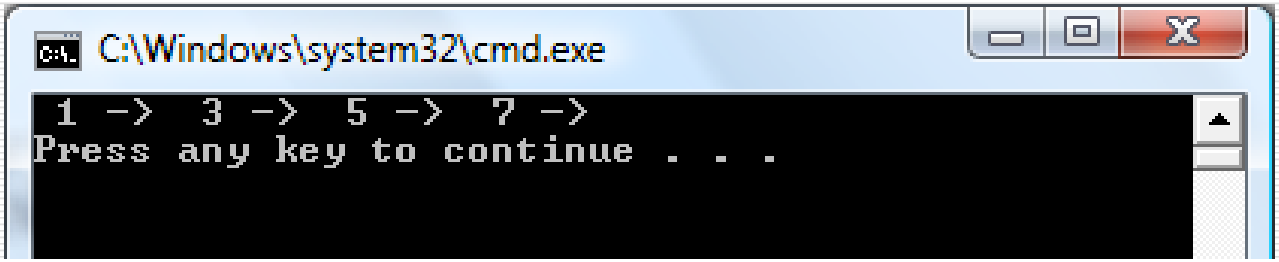
בדיקה

```
void main()
{
    int ar[]={1,3,5,7}, ind;
    Item *lst=NULL;
    Item *pos=NULL;

    // build list
    for(ind=0; ind<4; ind++)
        pos=Insert(ar[ind],pos,&lst);
    ...
```

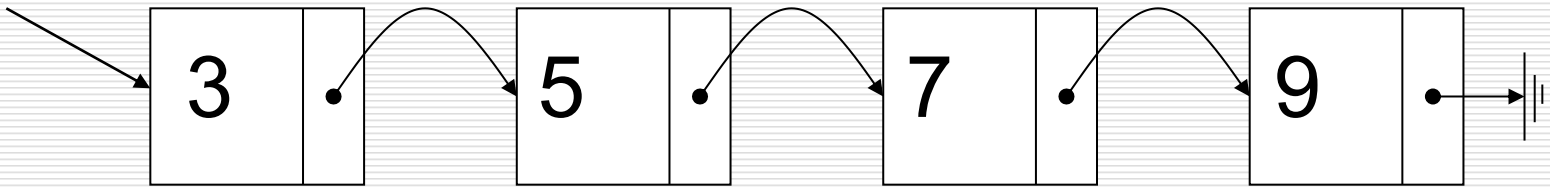
הדפסה

```
...
// print list
pos=lst;
while(pos)
{
    printf(" %d -> ", pos->data );
    pos=pos->next;
}
```



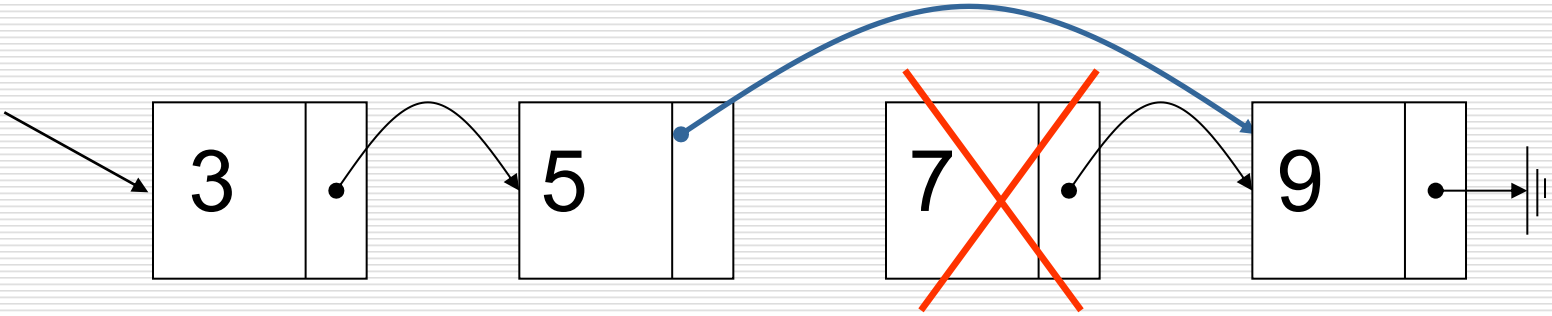
עוד דוגמא בציור: מחיקת איבר

- נניח שנתונה הרשימה הבאה של מספרים שלמים:



- נאמר שאנחנו רוצים למחוק את האיבר שבו נמצא המספר 7.
- אז צריך למצוא את מי שמצביע עליו (5), ולגרום לו להצביע על מי שאחרי 7 (כלומר 9).

רשימות מקושרות - מחיקת איבר



- נבצע הצבעה שעוקפת את 7, ונמחק את האיבר עם הערך 7.

- במערכים היינו צריכים "לצופף" את איברי המערך

מחיקת איבר

```
void Delete (Item *pos, list *pl)
{
    Item *temp;
    if (!(*pl)) return; // empty list
    if (!pos) // delete the first item
    {
       temp = *pl;
       *pl = temp->next;
       free (temp);
     }
    else  if (pos->next) // delete the second or other items
       {
         temp = pos->next;
         pos->next = temp->next;
         free (temp);
       }
}
```

מוחקים איבר שנמצא אחרי
איבר עליו מצביע pos

אם נמצא?

```
int isFound(list lst, list_type value)
{
    Item *check = lst;
    while (check != NULL)
    {
        if (check->data == value) return 1;
        check = check->next;
    }
    return 0 ;
}
```

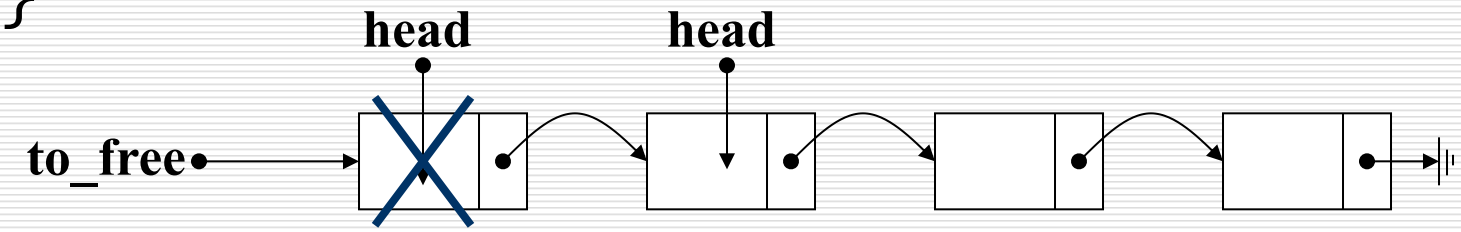
מציאת איבר ברשימה

המשימה מוצאת איבר שערכו value

```
Item *Find(list lst, list_type value)
{
    Item *check = lst;
    while ((check != NULL)  && (check->data != value))
        check = check->next;
    return check;
}
```

שחרור רשימה

```
void FreeList (Item *head)
{
    Item *to_free = head;
    while (to_free != NULL)
    {
        head = head->next;
        free(to_free);
        to_free = head;
    }
}
```

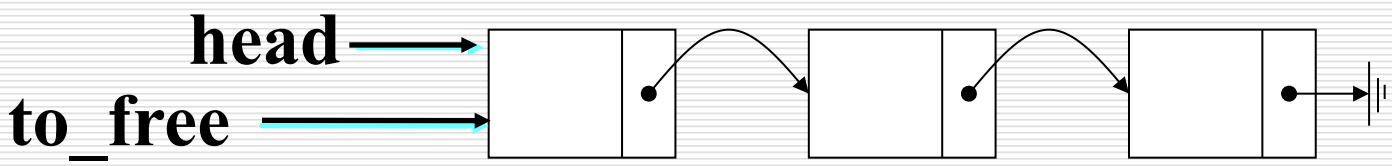


שחרור רשימה - מימוש

```
void FreeList (Item* head)
{
    Item* to_free = head;
    while (to_free != NULL)
    {
        head = head->next;
        free(to_free);
        to_free = head;
    }
}
```

מצביע נוסף על האיבר הראשון, שישוחרר

ממשיכים כל עוד לא הגענו לסוף הרשימה



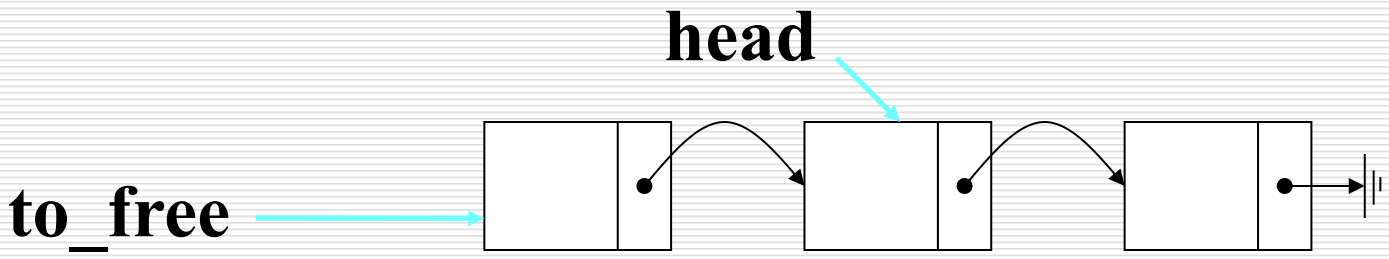
שחרור רשימה - מימוש

```
void FreeList (Item* head)
{
    Item* to_free = head;
    while (to_free != NULL)
    {
        head = head->next;
        free(to_free);
        to_free = head;
    }
}
```

מצביע נוסף על האיבר הראשון, שישוחרר

ממשיכים כל עוד לא הגענו לסוף הרשימה

ראש הרשימה מתקדם



שחרור רשימה - מימוש

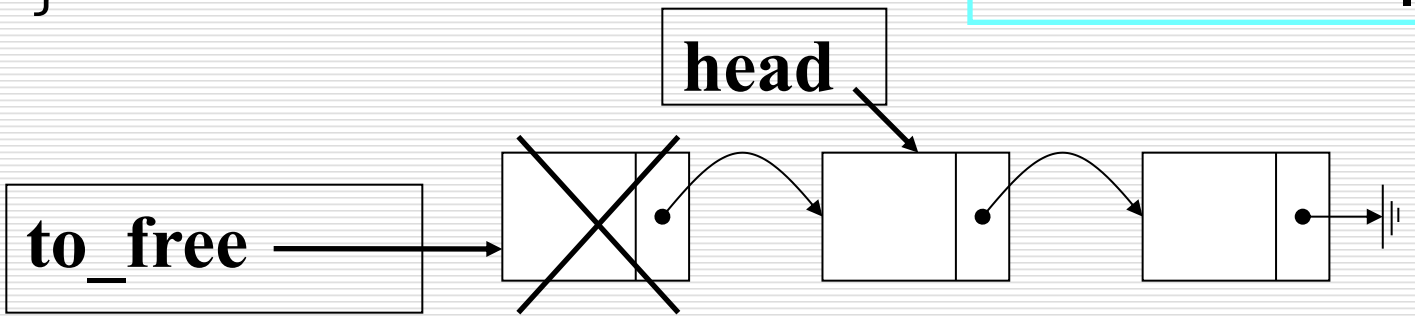
```
void FreeList (Item* head)
{
    Item* to_free = head;
    while (to_free != NULL)
    {
        head = head->next;
        free(to_free);
        to_free = head;
    }
}
```

מצביע נוסף על האיבר הראשון, שישוחרר

ממשיכים כל עוד לא הגענו לסוף הרשימה

ראש הרשימה מתקדם

האיבר הראשון משוחרר



שחרור רשימה - מימוש

```
void FreeList (Item* head)
{
    Item* to_free = head;
    while (to_free != NULL)
    {
        head = head->next;
        free(to_free);
        to_free = head;
    }
}
```

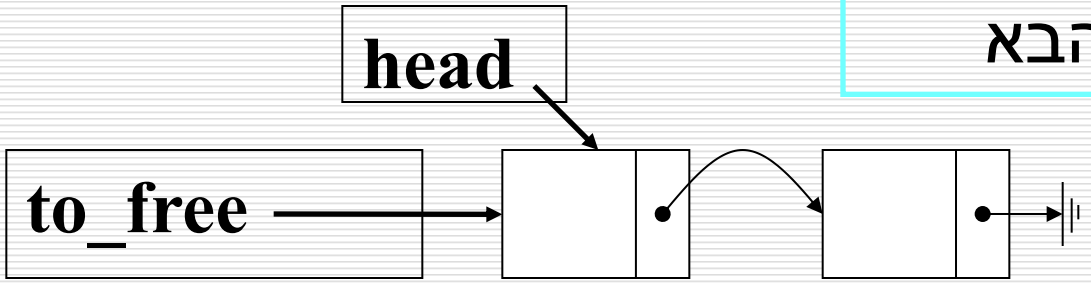
מצביע נוסף על האיבר הראשון, שישוחרר

ממשיכים כל עוד לא הגענו לסוף הרשימה

ראש הרשימה מתקדם

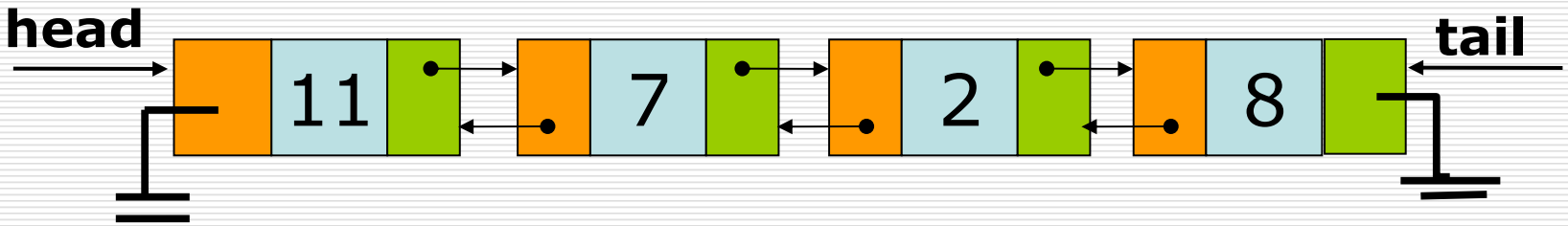
האיבר הראשון משוחרר

כעת ישוחרר האיבר הבא

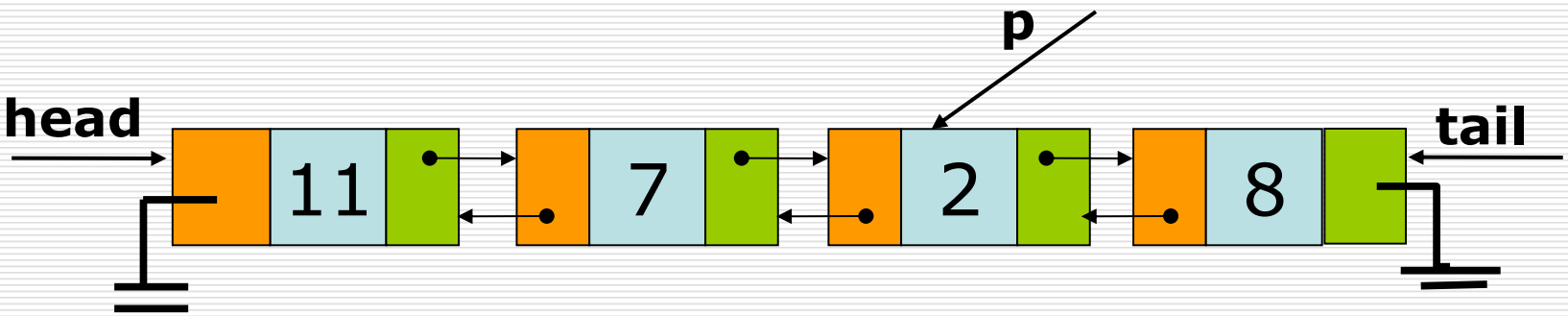


רשימות דו-כיווניות

```
typedef struct item
{
    list_type data;
    struct item *next, *prev;
}   Item;
```

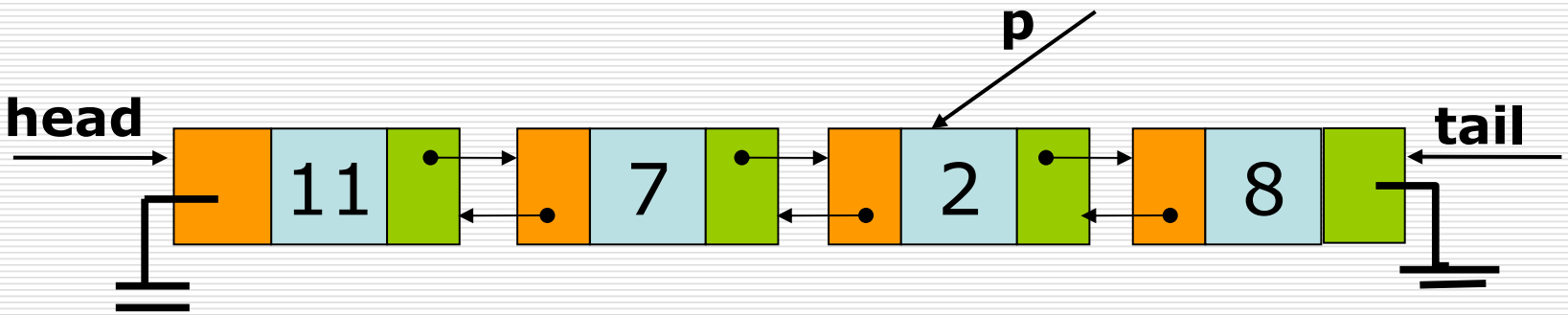


רשימות דו-כיווניות



- ניתן **להכניס** איבר חדש **אחרי** איבר שבכתובת **p** תוך זמן **O(1)**.
- ניתן **להכניס** איבר חדש **לפני** איבר שבכתובת **p** תוך זמן **O(1)**.

רשימות דו-כיווניות

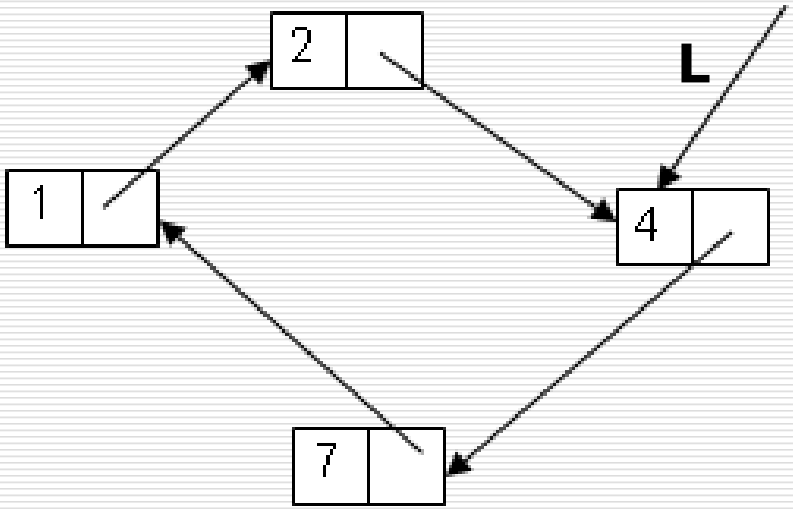


- ניתן **למחוק** איבר שבכתובת **p** תוך זמן **O(1)** וללא כתובת האיבר שלפניו.

```
p->prev->next = p->next;
p->next->prev = p->prev;
free(p);
```

רשימה מעגלית

- **רשימה מעגלית** היא רשימה בה האיבר **האחרון מצביע** לאיבר **הראשון**.



רשימות לעומת מערכים: יתרונות רשימה

- במהלך עבודה עם **רשימה** ניתן **לשנות גודל** שלה. **גודל מערך** (גם דינאמי) צריך **לקבוע** לפני תחילת עבודה אתו.

- הוספה ומחיקה של איברים מרשימה ידרשו תמיד רק שינוי מקומי (באיבר אחד או שניים), בעוד שבמערכים עשוי להידרש שינוי בכל המערך (אם בכלל יש מקום).

ז"א זמני ריצה של **הוספה ומחיקה** הם **O(1) ברשימה** ו-**O(n) במערך**.

- ניתן לשמור **איברי רשימה** בזיכרון **לא ברציפות**. **למערך** חייב להיות **מקום רציף** בזיכרון.

רשימות לעומת מערכים: חסרונות רשימה

- אבל יש גם חסרונות:
  - אין גישה ישירה לכל איבר ברשימה (למשל כדי לגשת לאיבר המיליון נצטרך לעבור את כל האיברים שלפניו כדי לדעת איפה הוא). ז"א זמן ריצה של **פניה לאיבר** הוא **O(n) ברשימה** ו-**O(1) במערך**.
  - הוספת איבר תדרוש הקצאה דינאמית, מה שלוקח **יותר זמן לעומת גישה למערך** שהוקצה מראש.
  - **איבר של רשימה** מכיל גם מצביע ולכן תופס **יותר מקום מאיבר של מערך**.
- לכן נעדיף **רשימה מקושרת** רק כשיש **הוספות ומחיקות תכופות**.

תרגילים

1. כתבו פונקציה המקבלת רשימת מספרים שלמים ומדפיסה איבריה קטנים מממוצע של איבריה.

2. כתבו פונקציה אשר מקבלת רשימת מספרים שלמים, ומחזירה מספר איברים מתחלקים ב- 3.

3. כתבו פונקציה אשר מקבלת שתי רשימות מספרים שלמים ומחזירה רשימה חדשה של איברים משותפים.